Building and Securing a REST API

Assignment Report — MoMo Data Pipeline

Team	Team Nexus
Members	Ange Umukundwa · Hugue Ishimwe · Dan Gisa
GitHub	https://github.com/Umukundwaa/momo-data-pipeline
Language	Python (http.server — no frameworks)
Dataset	modified_sms_v2.xml — 1,691 MoMo SMS transactions

This report covers: API Security Introduction · CRUD Endpoint Documentation · DSA Comparison Results · Basic Auth Limitations & Alternatives

1. Introduction to API Security

An API (Application Programming Interface) is the bridge between a client — such as a web dashboard or mobile app — and the server that holds data. Because APIs expose data over the internet, securing them is one of the most critical responsibilities in software development.

1.1 Why API Security Matters

The MoMo Data Pipeline API exposes sensitive financial transaction data — amounts, phone numbers, balances, and timestamps. Without proper security, any person on the internet could read, modify, or delete this data. API security ensures that only authorized users can access specific resources and that data is protected both in transit and at rest.

Unauthorized Access	Anyone could read private transaction data	Basic Authentication on all endpoints
Data Tampering	Attacker modifies transaction records	Auth required for POST, PUT, DELETE

Credential Theft	Username/password intercepted	HTTPS recommended in production
Brute Force	Attacker guesses passwords	JWT with expiry recommended for production
Injection Attacks	Malicious data sent in request body	Input validation on all POST/PUT requests

1.2 Authentication vs Authorization

Authentication answers: Who are you? — This is what Basic Auth does. It verifies the username and password before allowing access to any endpoint.

Authorization answers: What are you allowed to do? — In a production system, different users would have different permissions. For example, a read-only user could call GET endpoints but not POST, PUT, or DELETE. Our current implementation grants full access to any authenticated user.

1.3 What is Basic Authentication?

Basic Authentication works by encoding the username and password in Base64 and including them in the HTTP Authorization header of every request.

```
Format: Authorization: Basic base64(username:password)
Example: Authorization: Basic YWRtaW46bmV4dXMxMjM=
Decoded: admin:nexus123
```

■ **Important: Base64 is NOT encryption — it is just encoding. Anyone who intercepts the request can decode it instantly. Basic Auth must always be used with HTTPS in production.**

2. API Endpoint Documentation

Base URL	http://localhost:8000
Auth Method	Basic Authentication
Credentials	username: your username | password: your_password
Content-Type	application/json

2.1 Endpoints Overview

Method	Endpoint	Description	Auth
GET	/transactions	List all transactions (default 50)	Yes

GET	/transactions/{id}	Get one transaction by ID	Yes
POST	/transactions	Add a new transaction	Yes
PUT	/transactions/{id}	Update an existing transaction	Yes
DELETE	/transactions/{id}	Delete a transaction	Yes
GET	/health	API health check	Yes

2.2 Endpoint Details

GET `/transactions` Returns a paginated list of all transactions

Query Parameters:

Parameter	Type	Default	Description
limit	integer	50	Number of records to return
type	string	None	Filter by type e.g. 'Incoming Money'

Request:

```
curl http://localhost:8000/transactions -u admin:nexus123
```

Response (200 OK):

```
{ "total": 1691, "showing": 50, "transactions": [ { "id": 1, "transaction_id":
"76662021700", "type": "Incoming Money", "amount": 2000.0, "fee": 0.0, "sender":
"Jane Smith", "receiver": null, "new_balance": 2000.0, "date": "10 May 2024
4:30:58 PM" } ] }
```

GET `/transactions/{id}` Returns a single transaction by ID

Request:

```
curl http://localhost:8000/transactions/1 -u admin:nexus123
```

Response (404 Not Found):

```
{ "error": "Not Found", "message": "Transaction with ID 9999 does not exist."
```

} POST `/transactions` Creates a new transaction record

Required Fields: type, amount

Request:

```
curl -X POST http://localhost:8000/transactions \ -u admin:nexus123 \
-H "Content-Type: application/json" \ -d '{"type":"Incoming
Money","amount":15000,"sender":"John Doe","fee":0}'
```

Response (201 Created):

```
{ "message": "Transaction created successfully.", "transaction": { "id": 1692,
"type": "Incoming Money", "amount": 15000.0, "sender": "John Doe", "fee": 0.0 }
}
```

PUT `/transactions/{id}` Updates fields of an existing transaction

Request:

```
curl -X PUT http://localhost:8000/transactions/1 \ -u admin:nexus123 \
-H "Content-Type: application/json" \ -d '{"amount": 9999, "fee": 100}'
```

Response (200 OK):

```
{ "message": "Transaction updated successfully.", "transaction": { "id": 1,
"amount": 9999.0, "fee": 100.0, ... } }
```

GET`/transactions/{id}` Permanently deletes a transaction record

Request:

```
curl -X DELETE http://localhost:8000/transactions/1 -u admin:nexus123
```

Response (200 OK):

```
{ "message": "Transaction 1 deleted successfully." }
```

2.3 Error Codes Summary

Code	Name	When It Occurs
200	OK	Request succeeded — GET, PUT, DELETE
201	Created	New transaction created — POST
400	Bad Request	Missing required fields or invalid data
401	Unauthorized	Wrong credentials or no Authorization header
404	Not Found	Transaction ID does not exist or wrong endpoint

3. DSA Integration: Linear Search vs Dictionary Lookup

To efficiently retrieve transactions by ID, we implemented and compared two different data structures and search algorithms on the full dataset of 1,691 MoMo transactions.

3.1 Algorithm Descriptions

Linear Search — O(n)

Scans through the transactions list from position 0 one by one until the target ID is found. The time it takes grows directly with the size of the dataset. For a dataset of 1,691 records, it may check up to 1,691 records in the worst case.

```
def linear_search(data_list, target_id): for transaction in data_list:
if transaction['id'] == target_id: return transaction return None
```

Dictionary Lookup — O(1)

Uses a Python dictionary (hash map) built as {id: transaction}. Python computes a hash of the key and jumps directly to the memory address where the value is stored — no scanning needed. Time stays constant regardless of dataset size.

```
transactions_dict = {t['id']: t for t in transactions_list} def
dictionary_lookup(data_dict, target_id): return data_dict.get(target_id, None)
```

3.2 Benchmark Results

Both algorithms were tested on 20 transaction IDs spread across the dataset. Each test was run 1,000 times and the average time was recorded in microseconds (∝s).

Transaction ID	Position	Linear Search (∝s)	Dict Lookup (∝s)	Speedup
1	Start of list	0.29	0.24	1.2x
50	Near start	2.05	0.14	14.5x
100	Early	3.78	0.07	52.3x
200	Early-middle	6.55	0.07	90.2x
300	Middle	9.83	0.08	122.1x
500	Middle	13.90	0.08	178.5x
700	Middle-late	21.50	0.08	277.0x
900	Late-middle	30.54	0.14	215.5x

1000	Late	27.11	0.08	344.0x
1300	Near end	32.45	0.08	412.1x
1500	Near end	37.12	0.08	469.6x
1691	End of list	41.35	0.08	497.8x
999	Random middle	24.15	0.10	244.6x
AVERAGE	—	21.54	0.11	198.9x

Key Finding: Dictionary lookup is on average 198.9x faster than linear search on the MoMo dataset of 1,691 transactions.

3.3 Why Dictionary Lookup is Faster

Linear search time grows proportionally with dataset size (O(n)). Looking up ID 1,691 requires checking all 1,691 records. As the dataset grows to 100,000 records, linear search becomes 100,000 times slower.

Dictionary lookup uses a hash map internally. Python computes a hash function on the key (the ID number) which maps it to a specific memory slot. The lookup is O(1) — it takes the same amount of time whether the dataset has 10 records or 10 million records.

Property	Linear Search	Dictionary Lookup
Time Complexity	O(n) — grows with dataset	O(1) — always constant
Space Complexity	O(1) — no extra memory	O(n) — stores all keys
Build Time	None needed	O(n) — built once upfront
Best Case	O(1) — target is first	O(1) — always
Worst Case	O(n) — target is last	O(1) — always
Dataset of 1,691	Checks up to 1,691 records	Checks exactly 1 slot
Dataset of 100,000	Checks up to 100,000 records	Checks exactly 1 slot

3.4 Other Data Structures to Consider

Data Structure	Complexity	Best Used When
Binary Search (sorted list)	O(log n)	List is pre-sorted and rarely updated
Binary Search Tree (BST)	O(log n)	Frequent insertions and deletions needed
Hash Table (Python dict)	O(1)	Fast exact-key lookups — our choice
Trie	O(k)	Searching by text prefixes or strings
Heap	O(log n)	Finding min/max values quickly

4. Basic Auth Limitations & Stronger Alternatives

4.1 Why Basic Auth is Weak

■ **Basic Auth is NOT encryption. It encodes credentials in Base64 — which anyone can decode in seconds. It should never be used without HTTPS.**

Weakness	Explanation	Level
Credentials sent every request	Username and password are included in every single HTTP request	HIGH
Base64 is not encryption	Anyone can decode YWRtaW46bmV4dXMxMjM= to admin:nexus123 instantly	HIGH
No token expiry	Once credentials are stolen they work forever until password is changed	HIGH
Man-in-the-middle attacks	Without HTTPS, credentials are visible to anyone on the network	CRITICAL
No granular permissions	One password gives full access — cannot restrict read-only users	MEDIUM
No audit trail	Cannot track which specific client made which request	MEDIUM

Password sharing	All users share the same password — revoking one revokes all	MEDIUM

4.2 Stronger Authentication Alternatives

Option 1: JWT — JSON Web Tokens

JWT is the most popular API authentication method today. Instead of sending username and password with every request, the user logs in once and receives a signed token. That token is sent with future requests.

```
How it works: 1. Client sends: POST /login {username, password} 2. Server returns: {token: 'eyJhbGciOiJIUzI1NiJ9...'} 3. Client sends: GET /transactions Authorization: Bearer eyJhbGciOiJIUzI1NiJ9... 4. Server verifies token signature — no password needed again
```

JWT Advantage	Description
Token Expiry	Tokens expire after set time (e.g. 1 hour) — stolen tokens become useless
No password sharing	Password sent only once at login — never again
User roles	Token contains claims: {'role': 'read_only'} — fine-grained permissions
Stateless	Server does not store sessions — scales to millions of users
Audit trail	Each token has a unique ID — track exactly which client did what

Option 2: OAuth2

OAuth2 is the industry standard used by Google, GitHub, Facebook, and Twitter. It allows users to grant third-party apps access to their data without sharing their password. It is more complex than JWT but provides the highest level of security and flexibility for enterprise applications.

OAuth2 Advantage	Description
Third-party login	Login with Google/GitHub — no new password needed

Scopes	Grant specific permissions: read-only, write, delete
Token refresh	Short-lived access tokens + long-lived refresh tokens
Industry standard	Supported by all major platforms and libraries
Revocation	Revoke access for specific clients without affecting others

Option 3: API Keys

API Keys are unique identifiers issued to each client application. Simpler than OAuth2 but much better than Basic Auth because each client gets their own key that can be revoked independently.

4.3 Comparison Summary

Method	Security	Complexity	Best For
Basic Auth	LOW — Base64 only	Very Simple	Development and testing only
API Keys	MEDIUM — unique keys	Simple	Server-to-server communication
JWT	HIGH — signed tokens	Moderate	Web and mobile apps
OAuth2	VERY HIGH — standard	Complex	Enterprise and third-party access

5. Conclusion

This assignment successfully implemented a fully functional REST API for the MoMo Data Pipeline system using plain Python without any external frameworks. The API parses 1,691 MoMo SMS transactions from an XML file, exposes them through five CRUD endpoints, and protects all routes with Basic Authentication.

The DSA comparison demonstrated conclusively that dictionary lookup (O(1)) is 198.9 times faster on average than linear search (O(n)) on our dataset. As data grows, this performance gap widens further — making proper data structure selection critical for production systems.

While Basic Auth was implemented as required by the assignment, the security analysis shows it should be replaced with JWT tokens in any production deployment. JWT provides token expiry,

role-based permissions, and eliminates the need to send credentials with every request — addressing all of Basic Auth's key weaknesses.

Data Parsing	■ Complete	1,691 transactions parsed from XML into JSON
API Implementation	■ Complete	5 CRUD endpoints working with plain Python
Authentication	■ Complete	Basic Auth with 401 for invalid credentials
API Documentation	■ Complete	Full docs in docs/api_docs.md
DSA Integration	■ Complete	Dict lookup 198.9x faster than linear search